

10 Kinds of People

Day 4 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“Users of binary have to stay in the zones marked with a zero”*
- ▶ *“People can move north, south, east or west”*

Programming interpretation:

- ▶ The grid is a graph where adjacent equal digits are connected.
- ▶ Precompute connected components once instead of searching for every query.
- ▶ Two queried cells are connected only if their component IDs match.

Textbook connection: CP4 4.2.5 Flood Fill

Core technique: Connected components on an implicit grid

- ▶ Cells are nodes in a grid graph.
- ▶ Edges exist only between orthogonally adjacent cells with the same digit.
- ▶ Queries ask whether two cells are connected through one digit type.

- ▶ Running a search for every query can time out.
- ▶ Precomputing connected component IDs makes every query constant time.
- ▶ The grid is implicit, so neighbors are generated with direction arrays.

- ▶ Read the grid and initialize $\text{component}[r][c] = -1$.
- ▶ Scan every cell; when it is unlabeled, start DFS with a new component ID.
- ▶ DFS labels all same-valued cells reachable by four-direction moves.
- ▶ For each query, convert coordinates from 1-based to 0-based.
- ▶ Compare component IDs and print binary, decimal, or neither.

- ▶ The DFS stack avoids recursion-limit issues on large regions.
- ▶ The component ID captures both reachability and digit value.
- ▶ If IDs differ, no valid path exists even if the endpoint digits match.

- ▶ Flood fill visits exactly the cells reachable from the start through same-valued moves.
- ▶ Every cell is assigned to exactly one maximal same-valued connected component.
- ▶ Two queried cells are connected exactly when their component IDs match.

Complexity: $O(RC + Q)$ time and $O(RC)$ memory.

- ▶ Boundary checks must happen before indexing the grid.
- ▶ Print binary for component value 0 and decimal for component value 1.

Source lines: 10kindsofpeople.py:37-59

```
import sys

def main():
    input = sys.stdin.buffer.readline

    rows, cols = map(int, input().split())

    grid = []

    for _ in range(rows):
        grid.append(input().strip().decode())
```

What this chunk does

- ▶ The first line gives the grid dimensions.
- ▶ Each row is stored as a string of 0 and 1 characters.
- ▶ The digit represents which type of traveler can use that cell.

Source lines: 10kindsofpeople.py:60-80

```
component = [[-1] * cols for _ in range(rows)]  
  
component_id = 0  
  
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up # down # left #  
right
```

What this chunk does

- ▶ `component[r][c]` records the connected-region ID for each cell.
- ▶ -1 means the cell has not been assigned yet.
- ▶ The four directions enforce non-diagonal movement.

Source lines: 10kindsofpeople.py:82-99

```
def dfs(start_row, start_col, current_id):  
    value = grid[start_row][start_col]  
  
    stack = [(start_row, start_col)]  
  
    component[start_row][start_col] = current_id
```

What this chunk does

- ▶ The DFS captures the digit value of the starting cell.
- ▶ A stack avoids Python recursion-depth problems on large regions.
- ▶ The starting cell is marked immediately to avoid duplicates.

Source lines: 10kindsofpeople.py:100-130

```
while stack:
    row, col = stack.pop()

    for dr, dc in directions:
        new_row = row + dr
        new_col = col + dc

        if new_row < 0 or new_row >= rows or new_col < 0 or new_col >= cols:
            continue

        if component[new_row][new_col] != -1:
            continue

        if grid[new_row][new_col] != value:
            continue

        component[new_row][new_col] = current_id
        stack.append((new_row, new_col))
```

What this chunk does

- ▶ Each popped cell checks its four neighbors.
- ▶ Out-of-bounds, already visited, and different-valued cells are skipped.
- ▶ Valid neighbors receive the same component ID and are explored later.

Source lines: 10kindsofpeople.py:132-149

```
for row in range(rows):  
    for col in range(cols):  
  
        if component[row][col] == -1:  
            dfs(row, col, component_id)  
  
            component_id += 1
```

What this chunk does

- ▶ The double loop visits every cell in the map.
- ▶ An unvisited cell starts a new connected component.
- ▶ Each component ID is used later for constant-time query answers.

Source lines: 10kindsofpeople.py:151-195

```
queries = int(input())

output = []

for _ in range(queries):
    r1, c1, r2, c2 = map(int, input().split())

    r1 -= 1
    c1 -= 1
    r2 -= 1
    c2 -= 1

    if component[r1][c1] != component[r2][c2]:
        output.append("neither")

    elif grid[r1][c1] == "0":
        output.append("binary")

    else:
        output.append("decimal")

sys.stdout.write("\n".join(output))
```

What this chunk does

- ▶ Kattis coordinates are converted from 1-based to 0-based.
- ▶ Different component IDs mean neither traveler type can get through.
- ▶ Matching IDs are labeled binary or decimal based on the starting cell's digit.

Solution file: `practice_contests/day_4/10kindsofpeople.py`

Problem: <https://open.kattis.com/problems/10kindsofpeople>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.